



Plateforme de gestion de clinique vétérinaire

Simple, rapide, pensée pour les cliniques, les praticiens et leurs clients.



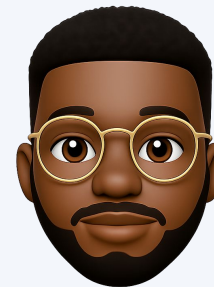
Jade



Mijosé



Cécile



Malick

6

rôles utilisateurs

3

applications

1

monorepo pnpm



Sommaire



01

Introduction

p.2



02

Gestion de projet

p.3



03

Démonstration & rôles

p.4–7



04

Technique

p.8



05

Sécurité

p.9



06

Infrastructure

p.10–12



07

Qualité

p.13



08

Observabilité

p.14–16



09

Résilience

p.17



10

Perspectives

p.18



Présentation du projet

01

Une plateforme de gestion de clinique vétérinaire, pensée sur le modèle de Doctolib

02

Simplifier la prise de rendez-vous, le suivi des animaux et la coordination des équipes soignantes

03

Un seul outil pour les clients, les praticiens et l'administration de la clinique

04

Développée en équipe de 4 dans le cadre du projet annuel M2 ESGI

6

rôles utilisateurs

3

applications — web, api, mobile

1

monorepo pnpm partagé



Construction du projet

01

Idéation

- Brainstorming général en équipe pour définir la vision du produit
- Liste de fonctionnalités détaillée, établie rôle par rôle

02

Wireframes

- Mise en œuvre visuelle des fonctionnalités identifiées
- Maquettes basse fidélité pour valider les parcours avant le développement

03

User stories

- Rédaction des user stories à partir des wireframes et des fonctionnalités
- Attribution des tâches à chaque membre et suivi de l'avancement du projet via Notion



Design system

01

Variables CSS pour les espacements, rayons et graisses de police

02

Thème Element Plus personnalisé — couleur primaire #256CAB

03

Typographies dédiées : Blue Winter pour la marque, Nunito pour les titres, DM Sans pour le texte

04

Composants réutilisables entre les rôles pour une cohérence visuelle totale

Palette



Primaire

#4873a2



Accent

#e06d84



Encre

#111C2C

Typographie

Blue Winter

Marque · logotype et accroches

Nunito — titres

DM Sans — texte courant et interface



• DÉMO & RÔLES

Six rôles, six expériences

01

Client

Prise de rendez-vous, suivi des animaux

02

Super Amin

Pilotage global de la plateforme

03

Directeur de clinique

Gestion de la structure et des équipes

04

Vétérinaire

Consultations, agenda, dossiers médicaux

05

Secrétaire

Accueil, planning, coordination

06

Référent clinique

Relais entre clinique et plateforme



Démonstration — Web & Mobile



Application Web

01

Vue 3 + Element Plus + Pinia + Vue Router

02

Calendrier interactif FullCalendar pour la gestion des rendez-vous

03

Tableaux de bord dédiés selon le rôle connecté

04

Navigation et layouts adaptés à chaque profil



Application Mobile

01

Développée avec Expo (React Native)

02

Accès simplifié pour les clients et les praticiens en mobilité

03

Suivi des rendez-vous et notifications

04

Base de composants partagée avec la logique métier commune



• DÉMO & RÔLES

Droit & RGPD

01

Données de santé animale et données personnelles des clients : une vigilance renforcée

02

Minimisation des données collectées à chaque formulaire et chaque rôle

03

Consentement et information claire lors de la création de compte

04

Droit d'accès, de rectification et de suppression des données personnelles

05

Journalisation des accès sensibles et séparation stricte des périmètres par rôle

OBJECTIF

Conformité

Une plateforme de santé exige un cadre juridique solide dès la conception —
privacy by design.



Choix de la stack technique



Web

- 01 Vue 3
- 02 Element Plus
- 03 Pinia
- 04 Vue Router
- 05 FullCalendar



API

- 01 Express 5
- 02 Prisma 7
- 03 PostgreSQL
- 04 Zod



Mobile

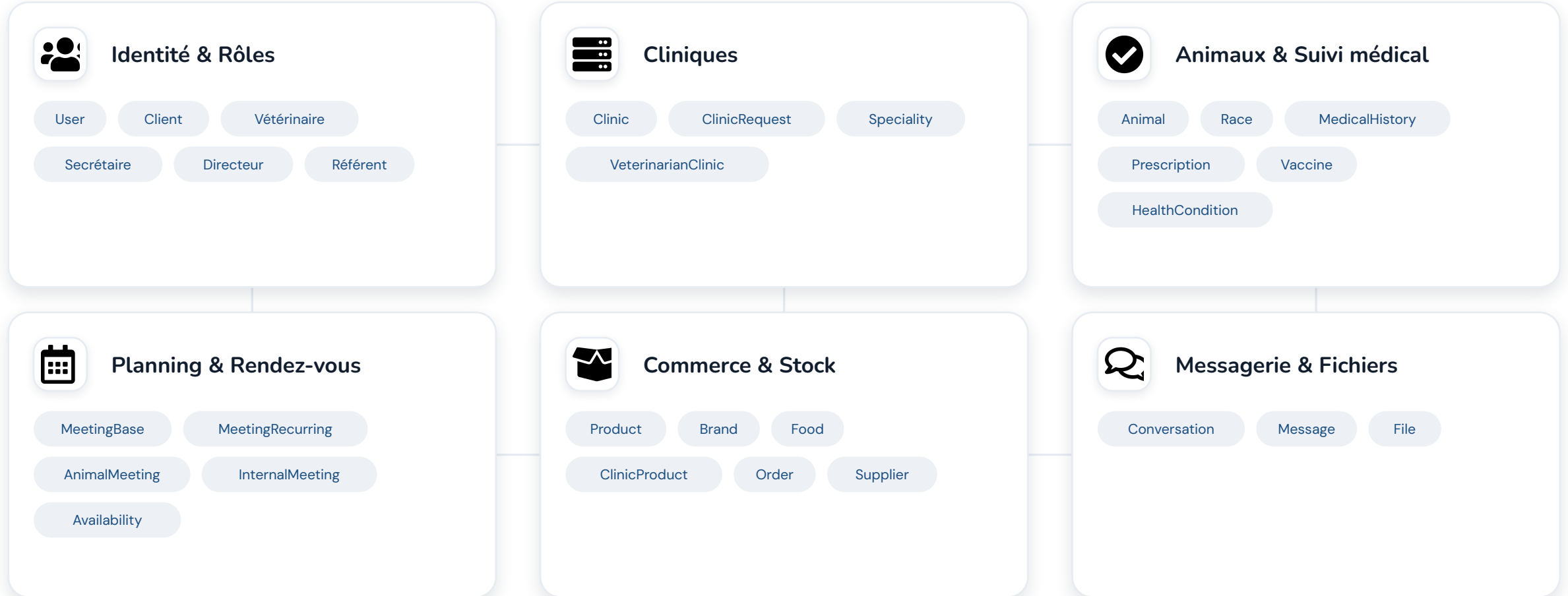
- 01 Expo
- 02 React Native

Monorepo pnpm — package partagé @armali/schemas (Zod) entre web, api et mobile



TECHNIQUE

Modèle de données



Relations Prisma (clés étrangères) entre les 45+ tables du schéma, regroupées ici par domaine fonctionnel



● SÉCURITÉ

Authentication

01

Authentication par JWT sur l'ensemble de l'API

02

req.user typé de façon optionnelle sur toutes les routes

03

AuthenticatedRequest utilisé explicitement sur les routes protégées

04

Middleware global de gestion des erreurs (hiérarchie AppError)

05

Contrôle des permissions par rôle à chaque endpoint sensible

Flux d'authentification

01

Login (email + mot de passe)

02

JWT émis par l'API

03

Requête + token en en-tête

04

Middleware d'authentification

05

Accès autorisé selon le rôle



Contrôle des accès - front

01

Hiérarchie de routes propre à chaque rôle, avec gardes d'accès dédiées

02

Layouts distincts

03

Permissions vérifiées côté client (navigation) et côté serveur (API)

04

Expérience cohérente quel que soit le profil connecté

Préfixes de routes par rôle

Client

/mon-espace

Vétérinaire

/veterinarian/...

Secrétaire

/secretary/...

Directeur de clinique

/director/...

Référent clinique

/referent/...

Super Admin

/admin/...



Docker — conteneurisation

01

Build multi-stage en production : dépendances → build → image finale allégée

02

En développement : sources montées en volume, hot-reload, sans étape de build

03

Prisma généré au build, migrations appliquées par un service dédié

04

Aucune valeur sensible dans les images — tout passe par la configuration Swarm

Configs & secrets Swarm

CONFIGS

prometheus.yml

grafana datasources

grafana dashboards

minio init script

SECRETS

db_user / password

jwt_access / refresh

stripe

resend

s3 / aws

minio root

matomo db

grafana admin

Secrets déclarés external: true — créés une fois sur le cluster, jamais versionnés dans le repo



Docker Swarm — architecture

01

Réplicas ajustés par service : 2 pour l'API, 3 pour le web et nginx

02

Mises à jour rolling (start-first) avec rollback automatique en cas d'échec

03

Services stateful contraints au nœud manager (Postgres, Minio, Matomo-DB)

04

Limites CPU/mémoire par service pour préserver la stabilité du cluster

Docker Swarm — OVH VPS



Traefik

Reverse proxy & TLS automatique (Let's Encrypt) · ports 80 / 443

armali.online

→ nginx

grafana.armali.online

→ grafana

matomo.armali.online

→ matomo

RÉSEAU FRONTEND

Traefik

Web

Nginx

API

Prometheus

cAdvisor

Grafana

Minio

Matomo

RÉSEAU BACKEND

Postgres

API

Minio

Matomo-DB

Matomo

Backup

Migration

Seed

API, Minio et Matomo sont rattachés aux deux réseaux



CI — Intégration continue

- 01 Déclenchée sur chaque pull request vers main ou dev
- 02 Une seule exécution active par PR (concurrency, annulation automatique)
- 03 Lint ESLint sur l'API et le Web, après build du package @armali/schemas
- 04 Couverture de code exigée uniquement pour les PR vers main

Pipeline CI

- 01 **Lint**
ESLint API + Web
- 02 **Test unitaire**
Prisma + TestContainers · couverture sur main
- 03 **Build**
Compilation API + Web

Node 24 · pnpm workspaces · cache pnpm activé sur chaque job



CD — Déploiement continu

01

Déclenché à chaque merge sur main

02

Terraform provisionne l'infrastructure de sauvegarde (OVH) avant tout déploiement

03

Build & push de 5 images (api, web, seed, nginx, backup) avec cache GitHub Actions

04

Déploiement SSH : docker stack deploy --with-registry-auth sur le cluster Swarm

Workflow de déploiement

01

Push sur main

02

Terraform — infra de sauvegarde

03

Build & push (5 images)

04

Sync compose + monitoring (SCP)

05

docker stack deploy (SSH)

Domaine armali.online · routage TLS géré par Traefik



Opérations à la demande

- 01 Migrations et seeds déclenchés manuellement (`workflow_dispatch`), hors du flux de déploiement
- 02 Le service cible est monté à 1 réplica, exécute sa tâche, puis redescend à 0
- 03 Les logs du service sont affichés directement dans le run GitHub Actions
- 04 Le seed en production exige une confirmation explicite : saisir « YES »

Migration & Seed

- 01 Pull de la dernière image API
- 02 Service scalé à 1 réplica (`--force`)
- 03 Exécution + lecture des logs (200 dernières lignes)

04 Service redescendu à 0 réplica

cecoule_migration et cecoule_seed restent à 0 réplica en permanence — jamais de service actif par défaut



Communication temps réel — WebSocket

01

Canal WebSocket dédié pour les mises à jour en temps réel (messagerie)

02

Diffusion ciblée par rôle et par clinique pour limiter le trafic

03

Reconnexion automatique côté client en cas de coupure réseau



Pourquoi ?

Une messagerie partagée entre secrétaires, vétérinaires et clients doit rester cohérente à l'instant T , sans rechargement manuel.



• QUALITÉ

Tests de l'API

01

Vitest comme framework de test sur l'ensemble de l'API

02

Près de 2 000 tests automatisés, couverture de code minimum 85 %

03

Tests de services : logique métier isolée, données mockées

04

Tests de routeurs : requêtes HTTP réelles sur une base de données de test seedée avec Prisma et Test containers

QUALITÉ

2 000+

tests Vitest

85 %

couverture min.

Services

Logique métier isolée, données mockées

Routeurs

Requêtes HTTP réelles sur une base de test seedée



• OBSERVABILITÉ

Prometheus & Grafana

01

Prometheus collecte les métriques applicatives et infrastructure

02

Grafana centralise les tableaux de bord de supervision

03

Suivi des performances API, de la charge des conteneurs et de la disponibilité

04

Base pour anticiper les montées en charge de la plateforme



Supervision

Métriques → collecte → visualisation : une boucle continue pour garder l'infrastructure sous contrôle.



• OBSERVABILITÉ

Sentry — suivi des erreurs

01

Capture automatique des exceptions côté web, API et mobile

02

Contexte complet à chaque erreur : utilisateur, rôle, requête

03

Alertes en temps réel pour réagir rapidement aux régressions

04

Historique des erreurs pour prioriser la dette technique



Réactivité

Détecter un incident avant qu'il ne soit remonté par un utilisateur.



• OBSERVABILITÉ

Matomo — analyse d'usage

01

Suivi de l'usage réel de la plateforme, respectueux de la vie privée

02

Analyse des parcours par rôle : pages consultées, taux de conversion

03

Données hébergées en interne, cohérentes avec les exigences RGPD

04

Aide à prioriser les évolutions produit sur des données concrètes



Usage produit

Comprendre comment chaque rôle utilise réellement la plateforme au quotidien.



Terraform — infrastructure de sauvegarde

01

Provisionne l'infrastructure de sauvegarde (bucket OVH, clés AWS DR) avant chaque déploiement

02

Exécuté automatiquement en amont du pipeline CD, avant le build des images

03

Élimine toute dérive de configuration entre les environnements de sauvegarde

04

Versionné avec le code applicatif pour une traçabilité complète



Objectif

Réduire le temps de reprise après incident et fiabiliser chaque redéploiement.



Évolutions futures

IDÉES PRODUIT

01 Nouveau rôle Gestion RH

Gestion poussée des contrats avec signature électronique, suivi des absences, congés et temps de travail.

02 Gestion des inventaires

Scan mobile des codes-barres produits pour dématérialiser l'inventaire médical.

APPROFONDISSEMENT TECHNIQUE

03 Autoscaling

Basé sur les métriques Prometheus déjà en place : scaling du nombre de replicas Docker Swarm en cas de forte demande.

04 Alerting Grafana

Alertes poussées vers Slack/Discord, au lieu de tableaux de bord uniquement consultés à la demande.

Armali



SOUTENANCE DE PROJET ANNUEL — M2 ESGI

Merci !

À bientôt sur Armali !



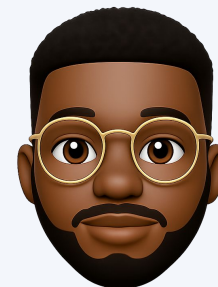
Jade



Mijosé



Cécile



Malick